ProSEIT

PROSEIT ANGULAR EXPERIENCE PLATFORM

PAEP

Implementation Architecture & Repository Baseline

ANGULAR WORKSPACE • PACKAGE BOUNDARIES • PRODUCT-NEUTRAL CONTRACTS • FRAPPE ADAPTER • RUNTIME CONFIGURATION • ANALYTICS & REPORTING • SECURITY • PORTABILITY • CI/CD

Field	Controlled Value
Document ID	PROSEIT-STD-PMRA-AF-006-IB01 | PAEP-IB-01
Version	1.0 | 14 September 2026
Status	Implementation baseline candidate derived from PAEP v1.1
Parent specification	ProSEIT Angular Experience Platform (PAEP) v1.1 - PROSEIT-STD-PMRA-AF-006
Solutions Architect & Main Product Owner	Dr Kamulegeya Grace B (PhD)
Developing Organisation	PROSEIT ORGANISATION OF SOFTWARE PROFESSIONALS LTD
Technology baseline	Angular 22.x active major; Angular CLI application builder; TypeScript 6.0.x; Nx-compatible workspace orchestration; npm workspaces; ERPNext/Frappe through typed adapters
Primary purpose	Turn PAEP v1.1 into a concrete developer implementation architecture and governed repository baseline.

CONTROLLED IMPLEMENTATION BASELINE

This document is subordinate to PAEP v1.1. It may make concrete implementation choices where PAEP v1.1 intentionally left tooling open, but it shall not weaken or reverse parent controls. Where any ambiguity exists, PAEP v1.1 prevails and the implementation baseline must be amended through architecture governance.

Document Control

Field	Controlled Value
Document title	ProSEIT Angular Experience Platform (PAEP) - Implementation Architecture & Repository Baseline
Document ID	PROSEIT-STD-PMRA-AF-006-IB01 / PAEP-IB-01
Version	1.0
Status	Implementation baseline candidate
Effective date	Upon Product Owner / Architecture approval
Parent architecture	PAEP v1.1 - PROSEIT-STD-PMRA-AF-006
Backend decision	ERPNext + Frappe retained; no backend replacement introduced
Repository posture	Governed Angular/Nx-style monorepo for PAEP platform packages and reference apps; independently releasable npm/Angular packages
Portability posture	Build once where practical; configure per deployment at runtime; no environment-specific source forks
Reporting posture	PAEP-owned reporting UX and contracts; governed operational reporting plus optional analytical store; external BI remains optional adapter.

Revision History

Version	Date	Disposition	Principal Change
1.0	14 Sep 2026	Baseline candidate	Initial concrete implementation architecture derived from PAEP v1.1. Defines toolchain, workspace, package boundaries,

Version	Date	Disposition	Principal Change
			contracts, runtime configuration, analytics/reporting contracts, reference app structure and CI/release gates.

Normative Relationship to PAEP v1.1

(i) Parent authority. PAEP v1.1 remains the architecture authority for modularity, security, accessibility, portability, backend authority, reporting governance, clean-room provenance and release evidence.

(ii) Concrete baseline. This document selects implementable defaults - including the initial Angular major, workspace tooling, package layering, runtime schema shape, state conventions and reporting interfaces - without altering those parent decisions.

(iii) Conflict rule. No implementation convenience may bypass PAEP contracts, backend authorization, tenant boundaries, accessibility gates, runtime-configuration controls, offline integrity controls or reporting governance.

(iv) Change rule. A change that alters a PAEP architecture principle or responsibility boundary must first be approved as a parent architecture amendment or ADR-backed exception before being incorporated here.

Table of Contents

1. Executive Summary
2. Purpose, Scope and Conformance
3. Controlled Implementation Decisions
4. Technology and Toolchain Baseline
5. Workspace and Repository Architecture
6. Package Boundaries and Dependency Rules
7. @proseit/paep-contracts Baseline
8. Platform Composition and Adapter Wiring
9. Runtime Configuration Schema and Bootstrap
10. State Management and Async Data Rules
11. Security and Browser Trust Implementation
12. Responsive UI/UX and Accessibility Implementation
13. Offline/PWA Implementation Baseline
14. Internationalisation and Localisation Baseline
15. @proseit/paep-analytics Implementation Architecture
16. Reporting Service and Semantic Contracts
17. Reference Application Structure
18. Testing, CI/CD and Release Engineering
19. Deployment and Portability Profiles
20. Observability and Supportability
21. Developer Workflow and Architecture Governance
22. Architecture Acceptance Gates
23. Initial Implementation Sequence
Appendix A. Full Repository Reference Tree
Appendix B. Package Dependency Matrix
Appendix C. Runtime Configuration Schema Excerpt
Appendix D. Reporting Contract Excerpt
Appendix E. Reference Application Route Baseline
Appendix F. Initial ADR Register
Appendix G. References

1. Executive Summary

This implementation baseline converts PAEP v1.1 from a controlled platform architecture into a concrete Angular workspace, package model and developer implementation contract. It preserves the central PAEP decisions: one governed experience platform; thin product applications; backend-authoritative domain logic and permissions; typed product-neutral contracts; ERPNext/Frappe behind an approved adapter; responsive and accessible interaction; bounded offline capability; runtime configuration; clean-room ownership; governed analytics; and evidence-backed releases.

The baseline deliberately separates architectural permanence from implementation currency. PAEP principles and package responsibilities are controlled architecture. Framework versions, compatible workspace tooling and third-party development dependencies are pinned in each PAEP release manifest so that the platform can upgrade without rewriting the architecture.

IMPLEMENTATION THESIS
Build PAEP as standard publishable Angular libraries and thin applications inside a governed monorepo; enforce dependency direction mechanically; keep product-neutral contracts independent of Frappe; wire concrete adapters at application bootstrap; configure deployments at runtime; and deliver Power BI-class reporting interactions through governed semantic reporting contracts rather than browser/database coupling.

(i) Modularity. Every package has an explicit responsibility, public API and dependency level. Product applications compose packages rather than copying them.

(ii) Scalability. Workspace orchestration, lazy/deferred loading, server-side data operations, bounded DOM rendering, asynchronous jobs and an optional analytical store allow the platform to scale across code, teams, data and deployments.

(iii) Security. The browser is never authoritative for permission, entitlement, fiscal, workflow or reporting security decisions. Runtime configuration contains no secrets, and production profiles enforce AOT, CSP, controlled Trusted Types and secure storage rules.

(iv) Configurability and portability. The same compiled application artifact is promoted across deployment environments where practical; endpoints, identity integration, telemetry and product options are supplied by a validated runtime configuration.

(v) Rich reporting. PAEP owns the dashboard/reporting user experience and semantic contracts. Operational reporting uses bounded domain/reporting APIs; historical and

cross-domain analytics may use a curated analytical store when justified. Power BI may be embedded for specific customers but is not a mandatory runtime dependency.

Figure 1. PAEP Implementation Architecture Baseline

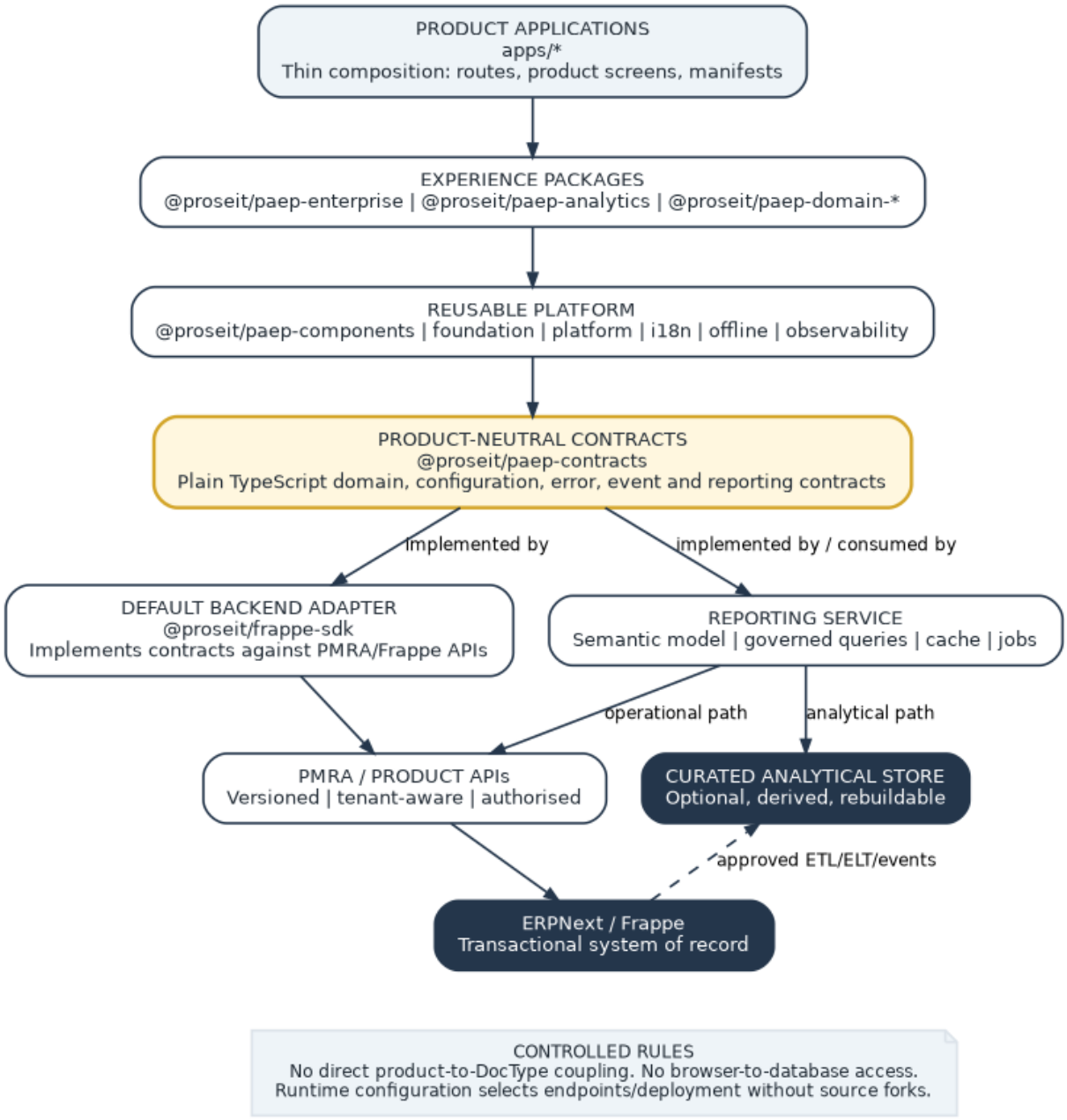


Figure 1. PAEP implementation architecture baseline. The contract boundary preserves portability while ERPNext/Frappe remains the strategic system of record.

2. Purpose, Scope and Conformance

2.1 Purpose

The purpose of PAEP-IB-01 is to give developers an implementation baseline that is specific enough to build, review, test and release the first PAEP platform without re-deciding core architecture in each product team.

2.2 In scope

(i) Angular workspace and repository structure, project naming, publishable package layout and public entry points.

(ii) Package dependency direction, architecture tags, prohibited imports and CI enforcement.

(iii) Product-neutral TypeScript contracts for configuration, errors, common domain concerns and reporting/analytics.

(iv) ERPNext/Frappe adapter placement and application-level dependency injection.

(v) Runtime configuration schema, startup validation, compatibility checks and fail-safe behaviour.

(vi) Frontend state ownership, async data, caching, realtime and offline interaction rules.

(vii) PAEP analytics/reporting components, semantic reporting contracts, operational versus analytical query paths, export jobs and optional external BI adapters.

(viii) Reference application structure, golden vertical slice, testing, build, SBOM, packaging and release evidence.

(ix) Cloud SaaS, private-cloud and on-premise deployment portability controls.

2.3 Explicit non-goals

(i) This baseline does not replace ERPNext/Frappe or move authoritative business logic into Angular.

(ii) This baseline does not make Nx, a visualization library or any other development tool an immutable architecture dependency. The selected tools are controlled implementation defaults that may be replaced through an ADR while preserving PAEP boundaries.

(iii) This baseline does not expose arbitrary SQL, DocType persistence or analytical-store credentials to browser code.

(iv) This baseline does not require every product to deploy a data warehouse/lakehouse. The analytical store remains conditional on evidenced workload drivers.

(v) This baseline does not attempt to clone Power BI, SAP Fiori, Ultima, PrimeNG or any proprietary screen implementation. PAEP implements capabilities and interaction requirements independently.

3. Controlled Implementation Decisions

Decision	Baseline	Rationale / parent alignment
Workspace orchestration	Angular CLI workspace with Nx-compatible orchestration and project graph; npm workspaces and committed package-lock.	Supports team-scale builds and boundary enforcement without adding runtime coupling.
Angular release	Angular 22.x active major at baseline approval; exact minor/patch pinned in release manifest.	Current supported Angular line as of 14 Sep 2026; preserves PAEP rule that framework versions are release-managed, not permanent architecture.
Component model	Standalone-first components/directives/pipes.	Matches PAEP v1.1 Angular engineering profile and current Angular guidance.
State model	Signals/computed for local/derived UI state; RxJS for streams/realtime/cancellable async integration; server state remains authoritative.	Prevents global-store sprawl while retaining observable integration patterns.
Configuration	Versioned JSON Schema plus generated runtime validator; immutable RuntimeConfigStore after successful bootstrap.	Implements PAEP fail-safe runtime configuration and portability rules.
Contracts	@proseit/paep-contracts is plain TypeScript with no Angular/Frappe/DOM/chart dependency.	Provides the portable contract boundary introduced in PAEP v1.1.
Backend adapter	@proseit/frappe-sdk implements approved contracts and transport conventions against PMRA/Frappe APIs.	Retains the ERPNext/Frappe backend decision and prevents direct DocType coupling.
Reporting	@proseit/paep-analytics owns UI patterns; ReportingGateway and semantic contracts live in paep-contracts; backend Reporting &	Supports Power BI-class interaction without browser/database coupling or BI-vendor lock-in.

Decision	Baseline	Rationale / parent alignment
	Analytics Service owns query authorization, aggregation, caching and jobs.	
Visualization engine	Apache ECharts-compatible adapter is the initial default; the charting engine is kept behind internal PAEP analytics adapters and is not part of public component contracts.	Rich visualization with replaceability and licence/security review.
Unit testing	Angular CLI default Vitest baseline for new projects.	Uses current Angular default while keeping test interfaces portable.
E2E testing	Playwright baseline.	Cross-browser, accessibility and responsive workflow automation.
Library publication	Angular Package Format / partial-Ivy for published Angular libraries.	Supports versioned reuse by product repositories and aligns with Angular library guidance.

These decisions are implementation defaults, not permission to bypass architecture review. Replacement is acceptable only when public contracts, quality gates, clean-room evidence, security posture and product compatibility remain intact.

4. Technology and Toolchain Baseline

4.1 Framework release profile

Area	Baseline	Control
Angular	22.x active major; latest approved compatible patch pinned per release.	@angular/core and Angular CLI remain aligned in the controlled manifest.
Node.js	Node 24 LTS in the Angular-supported range.	Exact version pinned in .nvmrc/Volta/asdf or equivalent developer tooling.
TypeScript	6.0.x for Angular 22 compatibility.	strict=true; strictTemplates=true; no implicit any; production build treats type errors as failures.
RxJS	Supported 7.x line.	Used for integration streams/realtime/cancellation; not

Area	Baseline	Control
		as a substitute for simple local component state.
Build system	Angular application builder.	No deprecated webpack browser builder in new PAEP apps.
Package manager	npm workspaces + committed package-lock.	CI uses npm ci; lockfile drift is release-blocking.
Workspace orchestration	Nx-compatible project graph, caching and module boundaries.	Dev/build-time only; packages remain standards-based Angular/npm packages.
Unit tests	Vitest through Angular CLI.	Component/platform unit suites remain deterministic and headless by default.
E2E	Playwright.	Runs representative browser matrix and viewport profiles.
Accessibility automation	axe-core integration plus manual keyboard/assistive-technology evidence.	Automation supplements rather than replaces manual WCAG evaluation.
SBOM	SPDX machine-readable release evidence.	Generated from locked dependency graph and retained with release attestation.

4.2 Version management rule

(i) Exact Angular, Node, TypeScript, RxJS, Nx and build-tool versions are recorded in the PAEP release manifest and lockfile.

(ii) Only supported Angular majors may be used for a new PAEP major/minor release unless an approved exception exists.

(iii) Framework upgrades are executed in a dedicated branch, run through the full compatibility matrix, and include migration notes for consuming products.

(iv) Developer-preview or experimental Angular APIs are not baseline dependencies unless an ADR identifies the benefit, fallback and exit strategy.

5. Workspace and Repository Architecture

The PAEP source repository is a single governed monorepo for shared platform packages, the reference application and the UI laboratory. Product repositories consume versioned

PAEP packages unless a product has been explicitly approved to coexist in the workspace. This keeps shared-platform evolution centrally governed while allowing product release independence.

5.1 Root repository baseline

```
paep/
  apps/
    reference-app/
    ui-lab/
  packages/
    contracts/ foundation/ components/ enterprise/ analytics/ platform/
    offline/ i18n/ observability/ frappe-sdk/
    domain-manufacturing/ domain-quality/ domain-maintenance/
    domain-construction/ domain-commerce/ domain-fiscal/ domain-ai/ domain-iot/
  tooling/
    architecture-boundaries/ config-validation/ test-utils/ visual-regression/
    a11y/ analytics-test/ build-release/
  config/
    runtime-config.schema.json
    runtime-config.example.json
  docs/
    architecture/ adr/ reporting-metrics/ provenance/ licences/ migration/
  evidence/
    tests/ accessibility/ usability/ analytics/ performance/ portability/
    security/ sbom/ releases/
  package.json  package-lock.json  angular.json  nx.json  tsconfig.base.json
```

5.2 Repository ownership rules

(i) apps/*. Contain bootstrap configuration, route composition, product-specific feature pages and product manifests. They do not own reusable PAEP platform code.

(ii) packages/*. Expose versioned public APIs. A package may have internal implementation folders, but consuming packages import only the package entry point or explicitly published secondary entry points.

(iii) tooling/*. Contains development and CI tooling only. Nothing under tooling is required in the production browser bundle.

(iv) config/*. Contains schemas and example configurations. Environment-specific production configuration is deployed separately from source and contains no server secret.

(v) docs/*. Contains ADRs, architecture, metrics definitions, provenance and migration guidance.

(vi) evidence/*. Contains retained test, accessibility, security, performance, portability, analytics, SBOM and release evidence. Evidence is generated/attached by CI and release workflows rather than edited as application source.

6. Package Boundaries and Dependency Rules

Figure 2. Package Dependency Direction and Enforcement

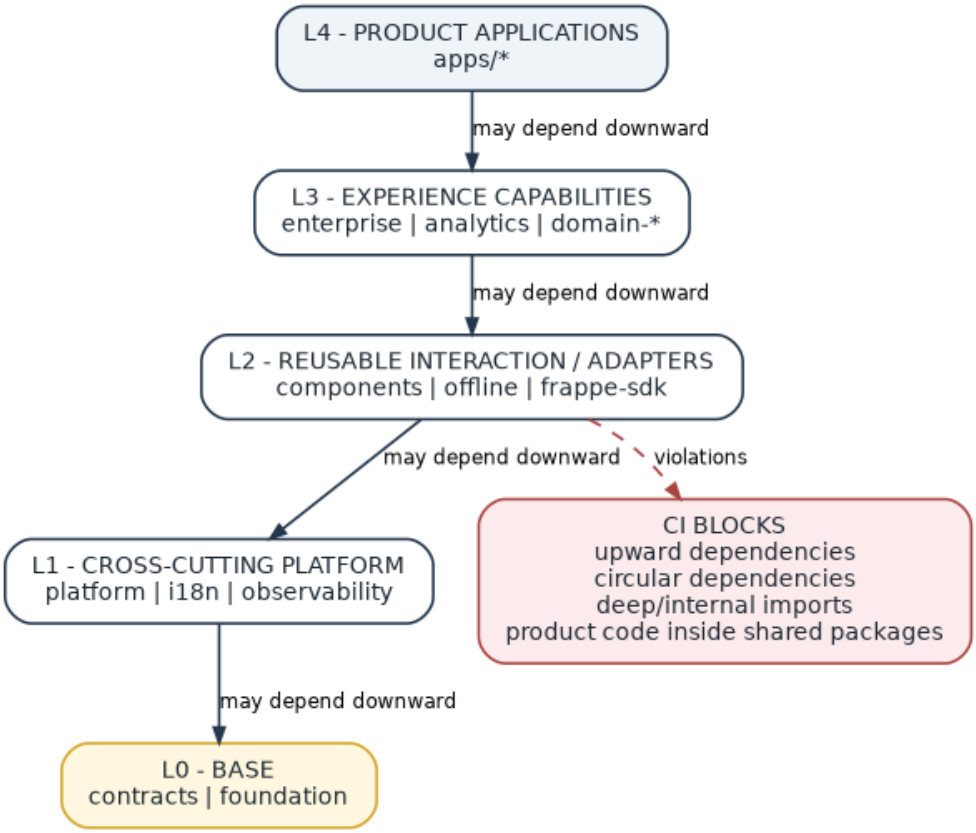


Figure 2. Allowed dependency direction. The enforcement model intentionally makes upward, circular and deep/internal imports mechanically detectable.

6.1 Layer definitions

Layer	Packages	May depend on
L0 - Base	@proseit/paep-contracts; @proseit/paep-foundation	Approved external dependencies only. contracts should remain framework-neutral; foundation may depend on Angular/CDK styling primitives but not product/platform packages.
L1 - Cross-cutting platform	paep-platform; paep-i18n; paep-observability	L0 only.
L2 - Reusable interaction / adapters	paep-components; paep-offline; frappe-sdk	L0-L1. frappe-sdk may depend on observability/contracts but never UI packages.

Layer	Packages	May depend on
L3 - Experience capabilities	paep-enterprise; paep-analytics; paep-domain-*	L0-L2 as required.
L4 - Applications	apps/*	Approved packages from L0-L3 and product-local features.

6.2 Mandatory dependency controls

(i) Circular package dependencies are prohibited and fail CI.

(ii) Deep imports such as @proseit/paep-components/src/internal/... are prohibited. Shared code is consumed only from public entry points.

(iii) @proseit/paep-contracts shall not import Angular, Frappe, DOM/browser APIs or visualization libraries.

(iv) @proseit/frappe-sdk shall not import PAEP UI or product packages.

(v) @proseit/paep-analytics shall not import Frappe persistence models or execute arbitrary transport calls. It depends on reporting contracts and injected gateway implementations.

(vi) Product applications may supply adapter providers but shall not reimplement platform contracts locally.

(vii) Domain packages may expose reusable interaction patterns but shall not embed authoritative business validation that belongs to backend/domain services.

(viii) Architecture tags and import rules are evaluated in lint/CI; exceptions require an ADR with an expiry or removal plan.

7. @proseit/paep-contracts Baseline

@proseit/paep-contracts is the key portability boundary introduced by PAEP v1.1. It defines what the frontend platform can ask for and what it can receive without knowing how ERPNext/Frappe, a reporting service, an identity provider or another backend implements the capability.

7.1 Contract package rules

(i) The package is plain TypeScript and must compile without Angular or browser globals.

(ii) Contracts use stable semantic names and avoid Frappe DocType field names unless a field has been deliberately promoted to a stable public domain contract.

(iii) Request/response models are readonly by default and versioned when semantics change.

(iv) Errors use controlled codes and field/global structures; UI code does not parse English backend prose.

(v) Every long-running operation exposes a job/status contract rather than relying on a blocking browser request.

(vi) Offline-safe write contracts identify idempotency/concurrency metadata where replay can create duplicate business effects.

(vii) Reporting contracts describe governed datasets, dimensions, metrics, filters, drill context, freshness and export jobs - never SQL.

7.2 Package structure

```
packages/contracts/src/
  lib/
    core/          # ids, paging, errors, correlation, capabilities
    config/        # RuntimeConfig and compatibility contracts
    identity/      # identity/tenant context and session outcomes
    domain/        # product-neutral workflow/file/list contracts
    reporting/     # dataset, metric, dimension, query/result, drill/export
    offline/       # queued-command and conflict metadata
  public-api.ts
```

7.3 Common contract conventions

Convention	Rule
Identifiers	Opaque string IDs at the UI boundary unless domain semantics require a stronger type.
Dates/times	ISO-8601 transport strings plus explicit business timezone context; locale formatting is a presentation concern.
Money	Transport amount plus currency code; never infer currency solely from browser locale.
Paging	Cursor-first for large/volatile collections; page-number support only where backend contract guarantees stable semantics.
Errors	Code + messageKey + optional fieldErrors + correlationId + retryable flag.
Concurrency	ETag/serverVersion or equivalent concurrency

Convention	Rule
	token on protected writes.
Idempotency	Client-generated operation key for replay-safe commands where duplicates would have business effect.
Capabilities	Server-derived capability identifiers; client flags do not create authority.

8. Platform Composition and Adapter Wiring

Applications assemble PAEP through dependency injection at bootstrap. Experience and domain packages depend on abstractions; the reference application binds those abstractions to the ERPNext/Frappe adapter and reporting gateway. This arrangement keeps the platform testable without Frappe and keeps product UI code independent of transport implementation.

8.1 Default provider composition

(i) Load and validate runtime configuration before protected routes are activated.

(ii) Provide RuntimeConfigStore as immutable application configuration.

(iii) Provide identity/session and active tenant context using @proseit/frappe-sdk for the baseline deployment.

(iv) Provide approved domain clients from @proseit/frappe-sdk against versioned PMRA/product APIs.

(v) Provide ReportingGateway against the Reporting & Analytics Service endpoint declared in runtime configuration; the default transport implementation may reside with the Frappe integration layer where that service is exposed through the same PMRA API boundary.

(vi) Provide observability, i18n, secure storage/platform abstractions and offline services before route composition.

(vii) Compose routes/navigation from the product manifest plus server-derived capabilities and entitlements. Client composition may hide or explain unavailable features but never grants permission.

8.2 Adapter testability

Every adapter contract shall have a test double in tooling/test-utils. Component and analytics packages are verified against those contract doubles independently of a live

Frappe backend. Separate conformance tests verify that @proseit/frappe-sdk produces the same contract behaviour against the reference backend.

9. Runtime Configuration Schema and Bootstrap

Figure 3. Runtime Configuration and Safe Bootstrap Sequence

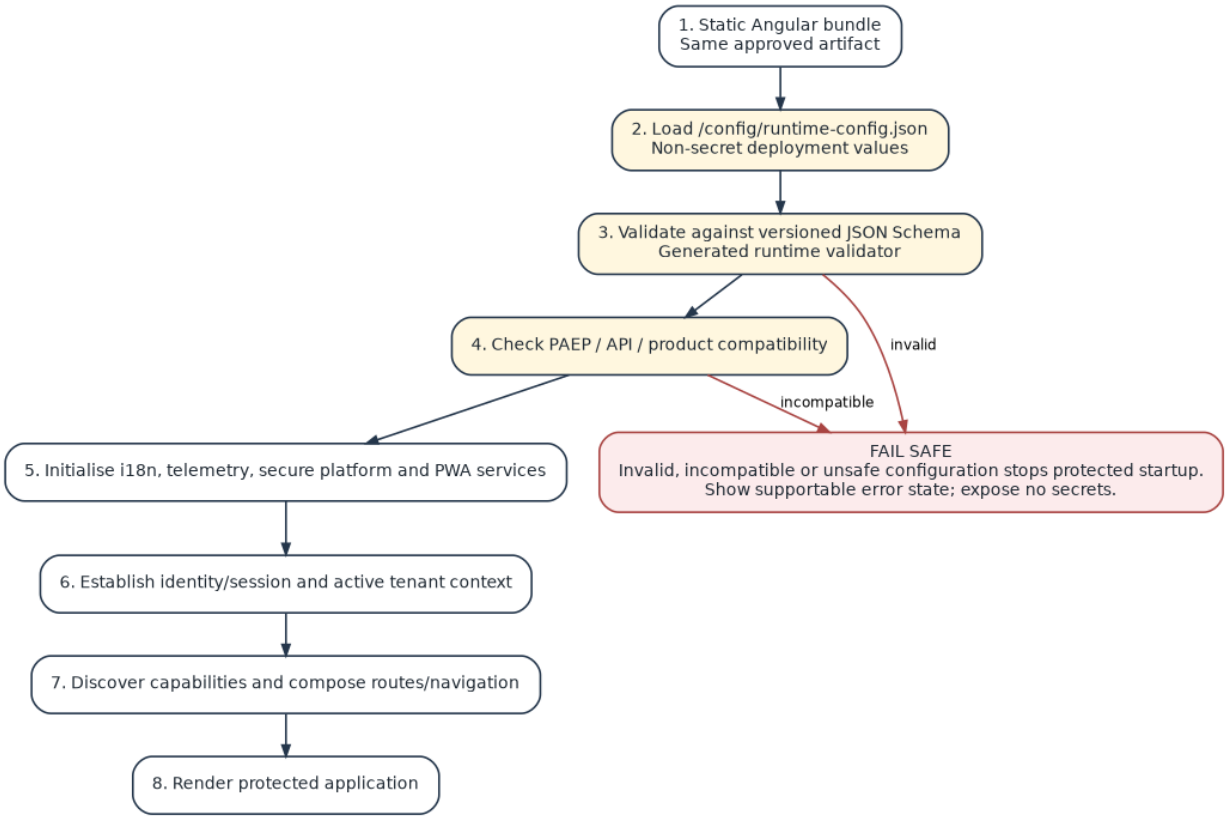


Figure 3. Safe startup ensures deployment configuration is validated before protected application behaviour is available.

9.1 Canonical configuration file

The canonical deployment artifact is /config/runtime-config.json, validated against config/runtime-config.schema.json. The runtime file is served alongside the static application and may differ by environment without rebuilding the application bundle.

Configuration area	Examples	Security / portability rule
schemaVersion	1.0	Unknown schema versions fail safely unless an approved backward-compatible parser exists.
product	id, name, release, brandProfile	Brand/product identity only; no entitlement authority.

Configuration area	Examples	Security / portability rule
deployment	environment, profile, basePath	Supports cloud-saas, private-cloud and on-premise without source forks.
backend	apiBaseUrl, realtimeUrl, fileBaseUrl, compatibility	URLs are non-secret. Compatibility is checked before unsafe use.
identity	mode, issuer, clientId, scopes	No client secret/private key. Server/session remains authoritative.
localisation	defaultLocale, supportedLocales, rtlLocales	Product activation only; reusable packages remain locale-ready.
features	capabilityDiscovery, uiHints	Hints affect presentation only; authority comes from the server.
telemetry	enabled, endpoint, sampleRate	No credential secrets; payloads remain privacy-controlled.
analytics	enabled, reportingBaseUrl, optional embeddedBi descriptor	No BI access token or database credential in static config. Tokens are obtained through backend-authorised flows.
pwa	enabled, offlineProfile	Offline profile defines browser capability; server rules remain authoritative.
security	trustedTypesPolicyName, cspProfile	Profile identifier only; deployment headers enforce actual CSP.

9.2 Startup validation and fail-safe behaviour

(i) CI validates all checked-in/example deployment configurations against the canonical JSON Schema.

(ii) The production bundle uses a generated standalone validator derived from the same schema so runtime validation does not depend on dynamic schema compilation.

(iii) RuntimeConfigStore is immutable after successful initialization. A remote configuration refresh, if ever introduced, must be a separately governed capability with change control.

(iv) Invalid JSON, schema failure, incompatible PAEP/API versions or an unsafe mandatory setting prevents protected route activation.

(v) The fail-safe screen provides a non-sensitive configuration/error identifier and support route; it does not expose secrets, stack traces or full internal endpoint payloads.

(vi) Compatibility checks are observable and included in support diagnostics.

9.3 Example runtime configuration


```
{
  "schemaVersion": "1.0",
  "product": {
    "id": "reference-app",
    "name": "PAEP Reference Application",
    "release": "0.1.0",
    "brandProfile": "proseit-default"
  },
  "deployment": {
    "environment": "development",
    "profile": "cloud-saas",
    "basePath": "/"
  },
  "backend": {
    "apiBaseUrl": "https://api.example.test",
    "realtimeUrl": "https://realtime.example.test",
    "fileBaseUrl": "https://files.example.test",
    "compatibility": {
      "apiContract": "pmra-api-v1",
      "minimumPaep": "1.1",
      "maximumPaep": null
    }
  },
  "identity": {
    "mode": "frappe-session",
    "issuer": null,
    "clientId": null,
    "scopes": []
  },
  "localisation": {
    "defaultLocale": "en",
    "supportedLocales": [
      "en",
      "fr",
      "pt",
      "ar"
    ],
    "rtlLocales": [
      "ar"
    ]
  },
  "features": {
    "capabilityDiscovery": "server",
    "uiHints": {
      "commandPalette": true,
      "analyticsWorkspace": true
    }
```

```
  },
  "telemetry": {
    "enabled": true,
    "endpoint": "https://telemetry.example.test",
    "sampleRate": 0.1
  },
  "analytics": {
    "enabled": true,
    "reportingBaseUrl": "https://reports.example.test",
    "embeddedBi": {
      "enabled": false,
      "provider": "power-bi"
    }
  },
  "pwa": {
    "enabled": true,
    "offlineProfile": "bounded-commands"
  },
  "security": {
    "trustedTypesPolicyName": "paep",
    "cspProfile": "strict"
  }
}
```

10. State Management and Async Data Rules

PAEP uses state locality rather than one application-wide mutable store. The state mechanism is selected by ownership and lifecycle, while backend state remains authoritative.

State category	Baseline mechanism	Rule
Pure local UI	signal() / component fields	Owned by the component; no global publication unless another feature actually needs it.
Derived UI state	computed()	Derive rather than duplicate mutable state.
Feature/workflow UI	feature-scoped signal store/service	Lives with the feature/domain package; released when feature context ends.
Server query state	typed client + feature query facade; optional resource/rxResource behind facade	Cache only within explicit freshness policy; never treat client cache as authority.
Realtime	RxJS observable/event stream	Reconnect performs safe

State category	Baseline mechanism	Rule
	mapped into feature state	refresh/reconciliation; events are freshness optimization, not sole truth.
Offline pending work	paep-offline queue state	Explicit local/queued/synced/conflict/failed states and idempotent replay rules.
Cross-application context	identity/tenant/product/config services	Small controlled set only. Tenant context changes invalidate incompatible cached state.

10.1 Global state library rule

NgRx or another global state framework is not a platform default. A product may introduce one only when an ADR shows that workflow complexity, event coordination or debugging requirements exceed the standard feature-local model. Even then, server state and security authority remain outside the client store.

11. Security and Browser Trust Implementation

11.1 Mandatory production controls

(i) AOT compilation is mandatory for production PAEP applications.

(ii) Production deployments enforce an approved Content Security Policy profile; application code shall not depend on permissive unsafe-inline/unsafe-eval behaviour.

(iii) Trusted Types are enabled where supported by the deployment/browser profile; bypassSecurityTrust* and direct innerHTML patterns require focused security review and tests.

(iv) Long-lived bearer credentials, server secrets, private keys and privileged service credentials shall not be stored in localStorage, IndexedDB or runtime configuration.

(v) Client validation improves UX only. All consequential writes, permissions, entitlements, workflow transitions, fiscal decisions and report-query security are validated server-side.

(vi) File previews/downloads use authorized URLs/tokens and controlled content types; client restrictions do not replace server scanning and authorization.

(vii) Source maps are handled according to release policy. If generated for diagnostics, deployment must prevent unintended public serving of source content.

(viii) Dependency lockfiles, vulnerability review, licences and SBOM form part of the release gate.

11.2 Tenant and cache isolation

(i) Active tenant/legal entity/project context is explicit in the shell and attached to authorized server requests as required by backend contracts.

(ii) Changing tenant context invalidates tenant-scoped query caches, offline views and report state that cannot be proven safe for the new context.

(iii) Analytics/report caches include every security-relevant dimension in cache keys. Shared server caches cannot return results across tenant/permission boundaries.

(iv) Logout, session expiry and device handover execute the approved secure-cache lifecycle while preserving only explicitly allowed local drafts/pending work.

12. Responsive UI/UX and Accessibility Implementation

PAEP treats responsiveness as preservation of task, context and meaning - not simply fluid width. Components and page patterns are implemented and tested at representative desktop, tablet and narrow-mobile profiles, with zoom/reflow, long translated text and keyboard navigation included in acceptance.

12.1 Adaptive interaction rules

(i) Critical content and actions must remain usable at 320 CSS-pixel reflow except where a genuinely two-dimensional visualization requires an accessible alternative.

(ii) Data tables use responsive strategies appropriate to the task: column priority, horizontal region scrolling, card/detail transformation, or drill-to-record. Whole-page horizontal scrolling is avoided for ordinary workflows.

(iii) Hover is never the only way to reveal required actions or explanations.

(iv) Touch target size and spacing are controlled design-system properties and remain usable across compact/comfortable density modes.

(v) Primary actions have stable placement within each page pattern; overflow menus are used for secondary actions rather than hiding the principal task.

(vi) Loading, empty, stale, permission-limited, offline, conflict and failure states use consistent platform patterns rather than ad-hoc screen messaging.

(vii) User preferences such as density, saved report view and selected columns never override permissions, required audit fields or accessibility semantics.

12.2 UX evidence

Critical product workflows require more than WCAG conformance. Release evidence records representative-user or approved equivalent usability evaluation, material errors/recovery problems and resolution of severity-1 usability findings. Analytics dashboards additionally measure whether users can identify current scope, freshness and drill context without ambiguity.

13. Offline/PWA Implementation Baseline

Offline capability is implemented only for explicitly approved workflows. The reference platform provides service-worker/update handling, connectivity state, queue primitives and secure local storage abstractions; products opt individual commands/workflows into offline execution.

State	UI meaning	Implementation requirement
Online/current	Confirmed server state within freshness target.	Normal server-authoritative workflow.
Local draft	Exists only on device.	Encrypted/protected according to profile; clear user disclosure; retention policy.
Queued	Committed locally but not yet accepted by server.	Idempotency key + immutable command envelope + tenant/user context metadata.
Synced	Server accepted the command.	Replace temporary client IDs with authoritative identifiers/version.
Conflict	Replay cannot be safely applied.	Retain attempted change and current server state; domain-specific resolution UI.
Failed	Retry/correction/escalation required.	Never imply success; correlation/support evidence retained.

(i) Service-worker updates shall not discard unsynchronized records.

(ii) Queued commands are encrypted/protected where required and partitioned by tenant and authenticated user context.

(iii) Logout/device handover warns about pending local-only work and follows product policy for retention or transfer.

(iv) Offline mode cannot create permission, entitlement or workflow authority; replay is still server-validated.

14. Internationalisation and Localisation Baseline

(i) All reusable user-facing strings use stable translation keys; product strings stay in product packages unless truly shared.

(ii) Locale-aware date, time, number, percentage and currency formatting is centralized in paep-i18n; business currency/timezone are explicit domain context, not inferred from browser locale.

(iii) Activated Arabic releases verify RTL layout, directional icons, tables, forms and reporting visual annotations.

(iv) Layouts tolerate text expansion. Critical labels are not truncated into ambiguity and fixed-width components are avoided where localization makes them unsafe.

(v) Report metric/dimension labels are supplied by semantic keys/metadata so dashboard definitions do not embed one-language prose.

15. @proseit/paep-analytics Implementation Architecture

@proseit/paep-analytics is the reusable Angular experience package for rich reporting. It is designed to achieve a Power BI-class decision experience - dashboards, scorecards, cross-filtering, drill, saved views, parameterized reporting, export jobs and governed self-service - without coupling PAEP to Power BI itself or exposing raw databases to the browser.

Figure 4. Rich Reporting Architecture - Operational and Analytical Paths

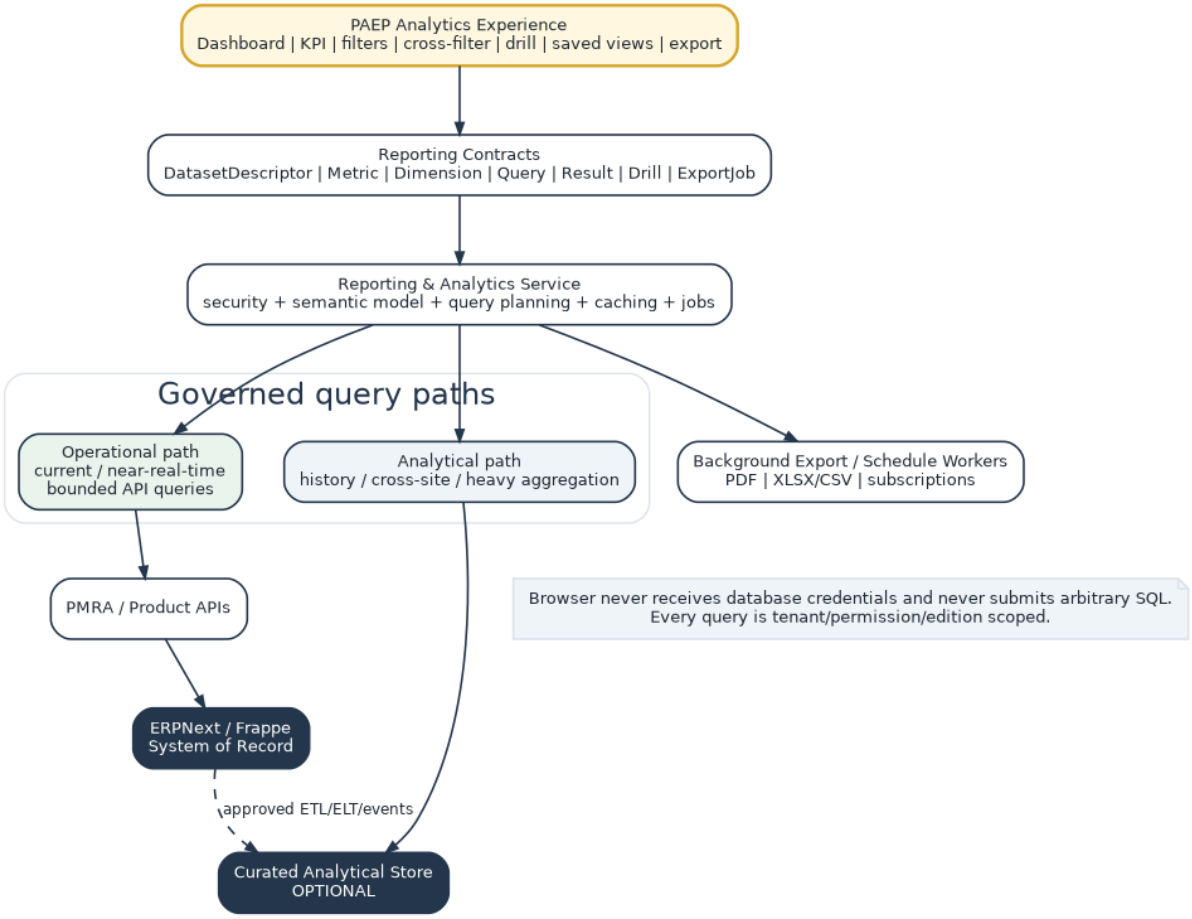


Figure 4. Rich reporting architecture. Operational and analytical paths are separated so dashboard sophistication does not compromise ERP transaction performance.

15.1 Initial reporting capability baseline

(i) Executive dashboards. KPI cards, trends, exception summaries, target/variance indicators and decision-oriented layouts.

(ii) Interactive filtering. Global/page/visual filters, date ranges, tenant/site/project context, explicit active-filter chips and predictable reset/apply behaviour.

(iii) Cross-filtering. Selecting a supported visual element can constrain peer visuals through a governed filter context rather than client-side raw-data joins.

(iv) Drill-down and drill-through. Hierarchies and detail targets are declared in the report definition. Drill context is preserved so the user can return to the originating analysis.

(v) Tabular analytics. Server-paged/sorted/filtered tables with column selection, pinned key columns, totals, export and accessible narrow-screen alternatives.

(vi) Saved views. User-scoped filter/layout preferences are stored through approved server/storage profiles and do not change report security or metric definitions.

(vii) Exports. CSV/XLSX/PDF exports are asynchronous jobs for large datasets; output authorization and expiry are server controlled.

(viii) Freshness. Every result includes as-of/source metadata. Hybrid dashboards can mix operational and analytical sources only when each visual declares freshness/source.

(ix) Self-service report designer. A later baseline phase may allow authorized users to choose governed datasets, dimensions, metrics, filters and visual types. It never exposes SQL or unrestricted schema browsing.

(x) Optional embedded BI. Power BI or another external BI tool may be integrated through an approved adapter for customer-owned BI scenarios; tokens and row security remain server-mediated, and embedded BI is not required for PAEP to function.

15.2 Visualization implementation boundary

The initial visualization adapter should use Apache ECharts or an equivalently approved open-source engine for charts because PAEP needs broad enterprise visualization without importing a proprietary template runtime. The library is an internal implementation dependency of paep-analytics: public PAEP components expose PAEP visual definitions, not ECharts option objects. This preserves replacement freedom and prevents product code from coupling to the chart engine.

15.3 Data-grid and pivot strategy

Power BI-class reporting does not require sending full analytical datasets to the browser. Grouping, pivoting, top-N, calculated measures and large aggregations are performed by the reporting service/analytical store. The browser receives bounded result sets and metadata. PAEP tables provide interaction, accessibility, virtualization and drill context; the server remains responsible for analytical scale and metric truth.

16. Reporting Service and Semantic Contracts

16.1 Semantic model responsibilities

Contract	Purpose	Authority
DatasetDescriptor	Names a governed reporting dataset and exposes available dimensions/metrics/freshness policy.	Reporting service / metric owner.

Contract	Purpose	Authority
DimensionDefinition	Defines categorical/time attributes, hierarchy and filter semantics.	Semantic model.
MetricDefinition	Defines controlled measures, type/format and description.	Metric owner; formula is server controlled.
ReportQuery	Requests metrics/dimensions/filters/sort/paging by semantic key.	Client may request; server validates every field/operator and security scope.
ReportResult	Returns bounded rows, schema, totals, freshness and correlation ID.	Reporting service.
VisualDefinition	Maps a semantic query to a PAEP visual type and drill/cross-filter behaviour.	Controlled dashboard/report definition.
DrillRequest	Carries governed source context to a declared target.	Server validates target and security.
ExportRequest / ExportJob	Starts and tracks asynchronous export.	Server authorizes, renders and controls download expiry.

16.2 Query-service rules

(i) Every query is evaluated with current identity, tenant/legal entity/project scope, permissions, edition/capability and dataset policy.

(ii) The service accepts semantic field keys and approved operators only; arbitrary SQL, unrestricted formulas and database table names are rejected.

(iii) Calculated metrics live in the semantic model/service so the same KPI definition is reused by dashboard, drill detail and export.

(iv) Operational queries use bounded PMRA/product APIs and must not create expensive unindexed ad-hoc scans over ERP persistence.

(v) Historical/cross-product workloads are routed to the analytical path when they meet the PAEP v1.1 analytical-store decision drivers.

(vi) Query plans enforce maximum time range, grouping cardinality, result size, execution timeout and export policy appropriate to the dataset.

(vii) Cache keys include semantic query, dataset version and all security-relevant context. Cached results retain source/freshness metadata.

(viii) Query audit records identify dataset/report, user/principal, tenant scope, time, high-level filter scope, result status and correlation ID without unnecessarily logging sensitive row data.

16.3 Analytical store baseline

The first PAEP deployment should begin with governed operational reporting APIs plus targeted materialized/aggregate backend views where appropriate. A separate analytical store is introduced only when historical retention, heavy aggregation, cross-site/product analytics, snapshot/time-series needs or reporting concurrency materially threaten transactional performance. When introduced, it is derived and rebuildable; ERPNext/Frappe and product domain services remain systems of record.

17. Reference Application Structure

Figure 5. Reference Application Composition

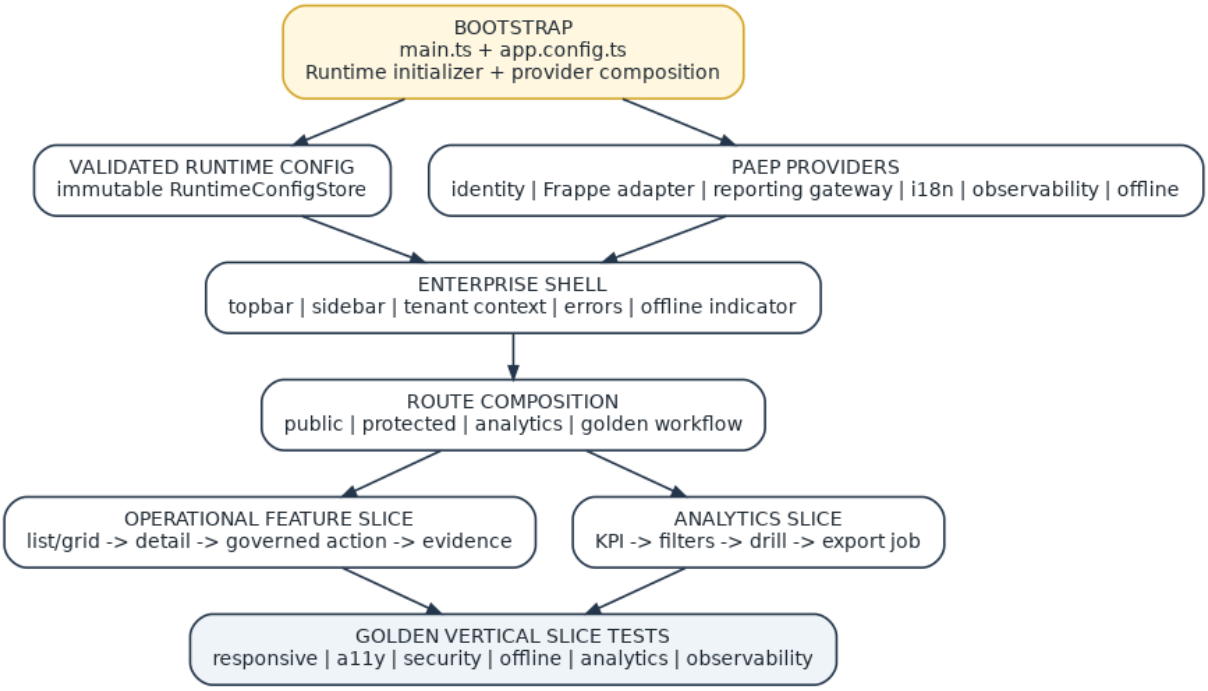


Figure 5. The reference application proves composition, adapter wiring, configuration, navigation and a golden workflow without becoming a product implementation.

17.1 Reference application responsibility

apps/reference-app is executable architecture evidence. It exercises the shared shell, runtime configuration, identity/tenant context, typed backend contract, analytics/reporting, a representative form/data grid/workflow action, file handling, responsive behaviour, accessibility, observability and a bounded offline scenario. It contains synthetic/reference domain content rather than production product logic.

17.2 Application folder baseline

```
apps/reference-app/src/app/
  app.config.ts
  app.routes.ts
  core/
    runtime/          # configuration initializer and compatibility gate
    auth/             # login/session presentation glue
    providers/        # PAEP adapter/provider composition
  features/
    workspace/        # golden operational workflow
    analytics/        # reference dashboard/report workspace
    evidence/         # file/evidence example
    offline-demo/     # bounded offline scenario
  layout/             # product-local shell composition only where needed
  testing/            # reference fixtures and route harnesses
```

17.3 Reference route baseline

Route	Purpose	Required evidence
/login	Identity/session flow.	Session failure/retry; accessibility; safe redirect.
/workspace	Golden operational workflow landing page.	Tenant context, responsive shell, data loading states.
/workspace/items	Data-heavy list/grid.	Server paging/filter, empty/error/permission states, responsive adaptation.
/workspace/items/:id	Detail + governed action.	Allowed-actions contract, conflict handling, audit result.
/analytics	Reference executive dashboard.	KPI, filter, cross-filter/drill, freshness, responsive/a11y.
/analytics/reports/:id	Parameterized report.	Semantic query contract, export job, permission-limited state.
/offline-demo	Bounded queued command.	Local/queued/synced/conflict/ failed states and recovery.
/support/diagnostics	Safe support metadata.	Release/config version, correlation identifiers; no secrets.

18. Testing, CI/CD and Release Engineering

Figure 6. CI/CD Quality and Release Pipeline

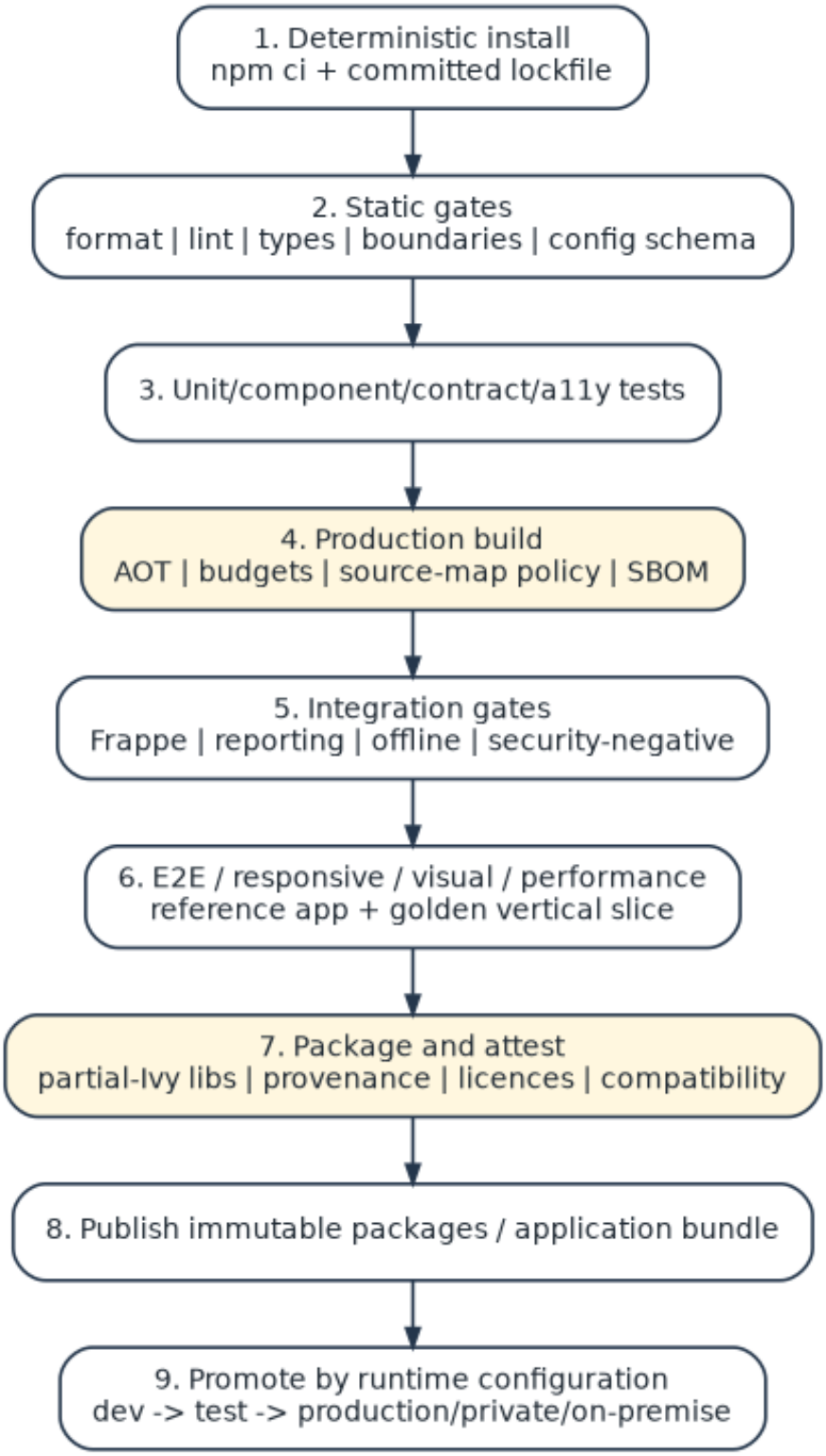


Figure 6. CI/CD quality gates from deterministic install through package publication and runtime-configured environment promotion.

18.1 Required CI gates

Gate	Minimum implementation evidence
Deterministic dependency gate	npm ci succeeds from committed lockfile; prohibited unapproved dependency changes fail review.
Static architecture gate	ESLint/types/strict templates; package-boundary tags; circular/deep-import checks; no direct product DocType imports.
Configuration gate	JSON Schema validation and negative startup tests.
Unit/component gate	Vitest suites for contracts mapping, state, components and analytics primitives.
Accessibility gate	Automated component/page checks plus retained manual keyboard/assistive-technology evidence for critical workflows.
Contract/integration gate	Frappe adapter conformance; API schema compatibility; reporting contract/metric tests; security-negative scenarios.
Offline gate	Replay/idempotency/conflict/update fault injection.
Visual/responsive gate	Representative breakpoints, activated locales/RTL and approved visual regression.
Performance gate	Bundle/route budgets; representative device/network; analytics interaction and data-volume tests.
E2E gate	Golden vertical slice across auth, tenant, workflow, evidence, reporting, errors and recovery.
Security/dependency gate	Vulnerability scan, CSP/Trusted Types deployment checks, source-map posture, SBOM and licence review.
Release attestation	Compatibility matrix, migration notes, provenance, evidence index and approvals.

18.2 Library publication

(i) Publish only approved package entry points; internal folders are not stable contracts.

(ii) Angular libraries are published in Angular Package Format using partial-Ivy compatible publication where distributed through the approved npm registry.

(iii) Contracts may be published as framework-neutral TypeScript packages and should not require Angular peer dependencies.

(iv) Semantic versioning applies to public PAEP APIs. Breaking changes require a major version or approved compatibility layer/migration window.

(v) Every release includes dependency/SBOM evidence, compatibility matrix and migration guidance.

19. Deployment and Portability Profiles

PAEP supports multiple hosting models without product source forks. The static frontend bundle, approved runtime configuration and backend endpoints form a deployment unit. Deployment-specific secrets remain on servers, identity providers, gateways or secret stores - never in the browser bundle.

Profile	Frontend deployment	Backend/reporting	Key controls
Cloud SaaS	Static assets through web host/CDN; runtime config per environment/region.	Shared/multi-tenant PMRA/Frappe and reporting services.	Tenant isolation, CDN origin policy, CSP, telemetry and release promotion.
Private cloud	Customer/partner cloud storage/web server/container; same PAEP build where practical.	Dedicated or private backend/reporting endpoints.	Configurable base path, identity/SSO, optional customer telemetry endpoint.
On-premise	Customer web server/reverse proxy/container or static host; self-hosted fonts/icons/assets.	On-premise Frappe/backend; optional local analytical store.	No mandatory external CDN/SaaS; controlled update process; local TLS/identity; offline network assumptions.

19.1 Portability acceptance rule

(i) Application source is identical across approved deployment profiles except where a formally approved platform capability requires a separate build target.

(ii) API, realtime, file, identity, analytics and telemetry endpoints are runtime configuration.

(iii) Base paths/subpath hosting are supported; hardcoded root-path assumptions are prohibited.

(iv) Core assets required for application use are self-hostable. External fonts, icons, analytics or CDNs are never mandatory to render protected workflows.

(v) External BI embedding is an optional deployment capability and is absent from the default runtime if not configured.

20. Observability and Supportability

(i) Every outbound backend/reporting operation propagates a correlation/trace identifier where supported.

(ii) Frontend error reports include release version, route, environment/profile, safe tenant context identifier, correlation ID and sanitized error category - not raw sensitive payloads.

(iii) Core Web/runtime metrics are collected according to privacy policy and sampling configuration.

(iv) Analytics observability distinguishes client-render time, query-service latency, cache state, data freshness and export-job duration so UI and backend bottlenecks can be separated.

(v) Support diagnostics expose the current PAEP version, product release, runtime schema/config identifier, API compatibility state and connectivity status without revealing secrets.

(vi) Telemetry adapters are replaceable so private/on-premise customers can disable or redirect telemetry without a source fork.

21. Developer Workflow and Architecture Governance

21.1 Development workflow

(i) Develop against the reference backend/test doubles through public contracts rather than importing product persistence details.

(ii) Create or amend an ADR before introducing a new cross-cutting dependency, global state framework, visualization engine, storage mechanism, micro-frontend boundary or deployment-specific source branch.

(iii) Add every reusable component to the UI laboratory with responsive, accessibility and state examples before promotion to platform stable status.

(iv) Add every governed metric/dimension to the reporting-metrics register with owner, definition, source, grain, security classification, freshness target and test fixture.

(v) Use generated/synthetic datasets in UI lab and analytics tests unless controlled test data has been approved.

(vi) Preserve clean-room provenance for design and code decisions and review third-party licences before dependency adoption.

21.2 Architectural exception rule

An implementation exception must state the violated baseline rule, architecture driver, scope, compensating controls, evidence, responsible owner and sunset/review date. Permanent divergence that changes PAEP principles is not an implementation exception; it requires amendment of the parent architecture.

22. Architecture Acceptance Gates

Gate	Acceptance evidence
Repository architecture	Workspace builds; project graph generated; public entry points defined; prohibited dependency tests green.
Contracts	paep-contracts compiles as plain TypeScript without Angular/Frappe/DOM/chart dependencies; contract fixtures retained.
Frappe integration	frappe-sdk conformance tests against reference backend; no arbitrary external DocType CRUD from product UI.
Runtime configuration	Schema + generated validator + negative bootstrap scenarios + compatibility check evidence.
Angular engineering	Standalone-first code, strict templates/types, zoneless compatibility review, lazy/deferred loading evidence, AOT production build.
Security	CSP/Trusted Types profile, browser-storage review, negative authorization tests, vulnerability/licence/SBOM evidence.
Responsive/a11y/UX	320px reflow where applicable, keyboard/focus, activated locale/RTL, screen-reader/manual evidence and critical workflow usability results.
Offline	Enumerated workflows, idempotency/conflict/retry/update tests and secure-cache lifecycle.
Reporting	Semantic dataset/metric contracts, tenant/row security, dashboard/filter/drill/export tests, freshness/source display and operational-load

Gate	Acceptance evidence
	isolation.
Portability	Same approved build demonstrated in required cloud/private/on-prem profiles or documented profile-specific build justification.
Release	Compatibility matrix, migration/rollback notes, provenance attestation and Product Owner/Solutions Architect acceptance.

23. Initial Implementation Sequence

The sequence below minimizes rework by building the contract and governance spine before visual breadth. Each phase exits with executable evidence.

Phase	Primary deliverables	Exit evidence
IB0 - Workspace	Angular 22 workspace, Nx/project graph, npm lockfile, CI shell, architecture tags/rules.	Clean build; boundary tests; deterministic install.
IB1 - Contracts/config	paep-contracts core/config/reporting; JSON Schema; runtime validator; test doubles.	Framework-neutral contract build; positive/negative config tests.
IB2 - Foundation/components	Tokens, typography, layout grid, action/form/data/overlay primitives.	UI lab; a11y/responsive/visual tests.
IB3 - Platform/enterprise	Runtime store, platform abstractions, i18n, observability, shell/navigation/error states.	Reference app boots from runtime config and passes shell tests.
IB4 - Frappe adapter	Session, typed list/detail/workflow/files/errors/realtime; compatibility checks.	Adapter conformance and backend integration tests.
IB5 - Analytics	Dataset/metric/query contracts, dashboard shell, KPI/chart/table/filter/drill/export UX.	Reference analytics dashboard with synthetic data; a11y/performance tests.
IB6 - Reporting service slice	Governed operational query endpoint, semantic model, export job; optional cache.	Metric consistency and tenant/permission-negative tests.
IB7 - Offline/PWA	Bounded command queue, sync/conflict/update handling.	Fault injection; no duplicate business effects.

Phase	Primary deliverables	Exit evidence
IB8 - Golden vertical slice	Auth + tenant + data grid + form + workflow + file + analytics + offline + observability.	End-to-end UAT/regression evidence.
IB9 - Product adoption	First real product consumes versioned packages and runtime config.	Compatibility, portability, migration and decommission evidence.

RECOMMENDED FIRST IMPLEMENTATION TARGET

Do not begin by building every domain kit. Build IB0-IB6 around the reference application and one real product vertical slice. This proves the architecture, reporting contracts and deployment model before the platform expands across the product portfolio.

Appendix A. Full Repository Reference Tree

```
paep/
├── apps/
│   ├── reference-app/
│   │   ├── public/config/runtime-config.json
│   │   └── src/app/{core,features,testing}/
│   └── ui-lab/
├── packages/
│   ├── contracts/src/lib/{core,config,identity,domain,reporting,offline}/
│   ├── foundation/
│   ├── components/
│   ├── enterprise/
│   ├── analytics/
│   ├── platform/
│   ├── offline/
│   ├── i18n/
│   ├── observability/
│   ├── frappe-sdk/
│   ├── domain-manufacturing/
│   ├── domain-quality/
│   ├── domain-maintenance/
│   ├── domain-construction/
│   ├── domain-commerce/
│   ├── domain-fiscal/
│   ├── domain-ai/
│   └── domain-iot/
├── tooling/
│   ├── architecture-boundaries/
│   ├── config-validation/
│   ├── test-utils/
│   ├── visual-regression/
│   ├── a11y/
│   ├── analytics-test/
```

```
│   └── build-release/
├── config/
│   ├── runtime-config.schema.json
│   └── runtime-config.example.json
├── docs/
│   ├── architecture/
│   ├── adr/
│   ├── reporting-metrics/
│   ├── provenance/
│   ├── licences/
│   └── migration/
├── evidence/
│   ├── tests/
│   ├── accessibility/
│   ├── usability/
│   ├── analytics/
│   ├── performance/
│   ├── portability/
│   ├── security/
│   ├── sbom/
│   └── releases/
├── angular.json
├── nx.json
├── tsconfig.base.json
├── package.json
└── package-lock.json
```

Appendix B. Package Dependency Matrix

From \ To	contracts	foundation	platform	i18n	observability	components	offline	frappe-sdk	enterprise	analytics	domain-*	apps/*
contracts	-	-	-	-	-	-	-	-	-	-	-	-
foundation	No	-	-	-	-	-	-	-	-	-	-	-
platform	Yes	Optional	-	-	Optional	-	-	-	-	-	-	-
i18n	Yes	Optional	Optional	-	Optional	-	-	-	-	-	-	-
observability	Yes	No	Optional	No	-	-	-	-	-	-	-	-
components	Optional	Yes	Optional	Yes	Optional	-	-	-	-	-	-	-
offline	Yes	No	Yes	Optional	Yes	No	-	-	-	-	-	-
frappe-	Yes	No	Option	No	Yes	No	Option	-	-	-	-	-

From \ To	contracts	foundation	platform	i18n	observability	components	offline	frappe-sdk	enterprise	analytics	domain-*	apps/*
sdk			al				al					
enterprise	Yes	Yes	Yes	Yes	Yes	Yes	Yes	No	-	-	-	-
analytics	Yes	Yes	Yes	Yes	Yes	Yes	Optional	No	Optional	-	-	-
domain-*	Yes	Yes	Optional	Yes	Optional	Yes	Optional	No	Optional	Optional	-	-
apps/*	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	-

Matrix interpretation: “Yes” means architecturally allowed when needed; “Optional” means allowed but should be justified by actual responsibility; “No” means prohibited; “-” means same package/self. Public entry-point and circular-dependency controls still apply even to allowed relationships.

Appendix C. Runtime Configuration Schema Excerpt

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://schemas.proseit.org/paep/runtime-config/1.0/schema.json",
  "title": "PAEP Runtime Configuration 1.0",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schemaVersion",
    "product",
    "deployment",
    "backend",
    "identity",
    "localisation",
    "features",
    "telemetry",
    "analytics",
    "pwa"
  ],
  "properties": {
    "schemaVersion": {
      "const": "1.0"
    },
    "product": {
      "type": "object",
      "required": [
        "id",
        "name",
        "release"
      ]
```

```
  },
  "deployment": {
   "type": "object",
   "required": [
    "environment",
    "profile",
    "basePath"
   ],
   "properties": {
    "profile": {
     "enum": [
      "cloud-saas",
      "private-cloud",
      "on-premise"
     ]
    }
   }
  },
  "backend": {
   "type": "object",
   "required": [
    "apiBaseUrl",
    "compatibility"
   ]
  },
  "identity": {
   "type": "object",
   "required": [
    "mode"
   ],
   "properties": {
    "mode": {
     "enum": [
      "frappe-session",
      "oidc",
      "sso-proxy"
     ]
    }
   }
  },
  "localisation": {
   "type": "object",
   "required": [
    "defaultLocale",
    "supportedLocales"
   ]
  },
  "features": {
   "type": "object",
   "required": [
    "capabilityDiscovery"
   ]
  },
  "telemetry": {
   "type": "object",
   "required": [
```

```
        "enabled"
      ]
    },
    "analytics": {
      "type": "object",
      "required": [
        "enabled"
      ]
    },
    "pwa": {
      "type": "object",
      "required": [
        "enabled",
        "offlineProfile"
      ]
    },
    "security": {
      "type": "object"
    }
  }
}
```

The complete machine-readable schema is supplied in the accompanying repository baseline at config/runtime-config.schema.json. The excerpt above is intentionally shortened for readability; the repository file is the implementation artifact validated by CI.

Appendix D. Reporting Contract Excerpt

```
import type { CorrelationId, PageInfo } from '../core/common';

export type FieldDataType = 'string' | 'number' | 'integer' | 'boolean' | 'date' | 'datetime' | 'currency' | 'percentage';
export type VisualType = 'kpi' | 'table' | 'bar' | 'line' | 'area' | 'pie' | 'scatter' | 'heatmap' | 'waterfall' | 'funnel' | 'combo';

export interface DimensionDefinition { key: string; labelKey: string; dataType: FieldDataType; hierarchy?: readonly
string[]; filterable?: boolean; }
export interface MetricDefinition { key: string; labelKey: string; dataType: FieldDataType; format?: string;
descriptionKey?: string; }
export interface DatasetDescriptor {
  id: string; version: string; labelKey: string;
  dimensions: readonly DimensionDefinition[];
  metrics: readonly MetricDefinition[];
  defaultTimeDimension?: string;
  freshnessPolicy?: { targetSeconds?: number; displayedToUser?: boolean; };
}
export type FilterOperator =
'eq' | 'neq' | 'in' | 'notIn' | 'gt' | 'gte' | 'lt' | 'lte' | 'between' | 'contains' | 'startsWith' | 'isNull' | 'isNotNull';
export interface ReportFilter { field: string; operator: FilterOperator; value?: unknown; }
export interface ReportSort { field: string; direction: 'asc' | 'desc'; }
export interface ReportQuery {
  datasetId: string;
  metrics: readonly string[];
```

```
  dimensions?: readonly string[];
  filters?: readonly ReportFilter[];
  sort?: readonly ReportSort[];
  cursor?: string;
  limit?: number;
  parameters?: Readonly<Record<string, unknown>>;
}
export interface ReportColumn { key: string; label: string; dataType: FieldDataType; format?: string; }
export interface DataFreshness { asOf: string; source: 'operational'|'analytical'|'hybrid'; stale?: boolean; }
export interface ReportResult<Row extends Readonly<Record<string, unknown>> = Readonly<Record<string,
unknown>>> {
  columns: readonly ReportColumn[];
  rows: readonly Row[];
  page?: PageInfo;
  totals?: Readonly<Record<string, unknown>>;
  freshness: DataFreshness;
  correlationId: CorrelationId;
}
export interface DrillRequest { reportId: string; visualId: string; sourceContext: Readonly<Record<string, unknown>>;
target: string; }
export interface ExportRequest { reportId: string; format: 'csv'|'xlsx'|'pdf'; query: ReportQuery; }
export interface ExportJob { id: string; status: 'queued'|'running'|'succeeded'|'failed'|'expired'; progress?: number;
downloadUrl?: string; expiresAt?: string; correlationId?: CorrelationId; }
export interface VisualDefinition { id: string; type: VisualType; titleKey: string; query: ReportQuery; crossFilter?:
boolean; drillTargets?: readonly string[]; }
export interface DashboardDefinition { id: string; version: string; titleKey: string; visuals: readonly VisualDefinition[];
globalFilters?: readonly ReportFilter[]; }

export interface ReportingGateway {
  getDataset(id: string): Promise<DatasetDescriptor>;
  execute(query: ReportQuery): Promise<ReportResult>;
  drill(request: DrillRequest): Promise<ReportResult>;
  requestExport(request: ExportRequest): Promise<ExportJob>;
  getExportJob(id: string): Promise<ExportJob>;
}
```

Appendix E. Reference Application Route Baseline

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'workspace' },
  { path: 'workspace', loadComponent: () => import('./features/workspace/workspace.page').then(m =>
m.WorkspacePage) },
  { path: 'analytics', loadChildren: () => import('./features/analytics/analytics.routes').then(m =>
m.ANALYTICS_ROUTES) },
  { path: '**', loadComponent: () => import('@proseit/paep-enterprise').then(m => m.NotFoundPage) },
];
```

(i) Product applications extend the route set through product manifests and feature packages; they do not edit PAEP enterprise internals.

(ii) Route guards may improve UX and prevent accidental navigation, but every server operation remains independently authorized.

(iii) Analytics routes lazy-load by default because charting/reporting code is not required for initial operational shell render.

(iv) Route-level errors and incompatible runtime/API versions resolve to controlled supportable states rather than blank screens.

Appendix F. Initial ADR Register

ADR	Decision
ADR-001	Angular 22 active-major implementation baseline and supported upgrade policy.
ADR-002	Nx-compatible workspace orchestration with npm packages; no runtime dependency on workspace tooling.
ADR-003	Product-neutral paep-contracts and Frappe adapter separation.
ADR-004	Signal-oriented local/derived state and server-authoritative data model.
ADR-005	Versioned runtime configuration schema and deploy-once/runtime-configure model.
ADR-006	Reporting semantic model, operational/analytical paths and analytical-store decision rule.
ADR-007	Default visualization engine adapter and replacement boundary.
ADR-008	Production AOT, CSP, Trusted Types and browser-storage profile.
ADR-009	Offline command queue, secure cache and idempotency model.
ADR-010	Package publication, partial-Ivy format and compatibility matrix.

Appendix G. References

[1] ProSEIT Organisation of Software Professionals Ltd, ProSEIT Angular Experience Platform (PAEP), Clean-Room Angular Design System & Enterprise Application Template Specification, Version 1.1, PROSEIT-STD-PMRA-AF-006, 14 Sep. 2026.

[2] Angular, Version compatibility, current documentation, consulted 14 Sep. 2026. Angular 22 is the active supported major; compatibility table defines supported Node.js and TypeScript ranges.

[3] Angular, Components / NgModules guidance, current documentation, consulted 14 Sep. 2026. New Angular code is standalone by default and standalone components are recommended for new code.

[4] Angular, Performance and deferred loading with @defer, current documentation, consulted 14 Sep. 2026.

[5] Angular, provideAppInitializer API, current documentation, consulted 14 Sep. 2026. Used for controlled asynchronous application startup such as runtime configuration loading.

[6] Angular, Testing overview, current documentation, consulted 14 Sep. 2026. New Angular CLI projects use Vitest by default.

[7] Angular, Creating libraries, current documentation, consulted 14 Sep. 2026. Published Angular libraries use the Angular Package Format and partial-Ivy guidance.

[8] W3C, Web Content Accessibility Guidelines (WCAG) 2.2.

[9] OWASP, Application Security Verification Standard (ASVS), as inherited by PAEP v1.1.

[10] SPDX, Software Package Data Exchange standard for machine-readable SBOM/provenance evidence.

REFERENCE CONTROL
This implementation baseline uses current Angular documentation only to select concrete implementation defaults consistent with PAEP v1.1. The parent specification remains the authority for ProSEIT architecture, security, accessibility, clean-room and product-governance controls.